

Understanding the structure the frontend will take / how the frontend will work

The three-page design

I can satisfy both **My design** and the **ticket requirements** without creating loads of separate pages.

Page 1 — Home /

This will be the landing page.

Styled similarly to the Encon site screenshots you supplied:

```
1 Dark Encon navy
2 White cards
3 Light blue/cyan accents
4 Large clean headings
5 Rounded navigation bar
6 Large spacing
7 Professional pharmacy/operations appearance
```

It will contain:

```
1 Cold Chain Operations Portal
2
3 Short explanation of the system
4
5 How it works:
6 1. Log a temperature reading
7 2. System checks 2°C-8°C
8 3. Excursions trigger the alert workflow
9 4. View readings and alert information
10
11 Navigation:
12 [ Manage Readings ]
13 [ View Readings ]
```

This isn't just decoration — someone opening the portal should immediately understand what they're supposed to do.

Page 2 — Manage Readings /manage

This is where we'll put the **write operations**.

Rather than having separate pages for POST/PATCH/DELETE, I suggest one management screen with three sections/tabs:

```
1
2 Manage Temperature Readings
3
4 [ Create ] [ Edit ] [ Delete ]
5
```

CREATE

Something like:

```
1 Branch
2 [ BRANCH-001 ]
3
4 Fridge
5 [ FRIDGE-001 ]
6
7 Recorded At
8 [ 08/09/2026 12:30 ]
9
10 Minimum Temperature °C
11 [ 3 ]
12
13 Maximum Temperature °C
14 [ 7 ]
15
16 Recorded By
17 [ Umar ]
18
19 [ Save Reading ]
```

The **UUID** will **NOT** be entered by the user.

That's important because your backend generates the UUIDv4 itself.

After POST:

```
1 POST
2 ↓
3 backend generates UUID
4 ↓
5 DynamoDB
```

If normal:

```
1 ✓ Reading saved successfully
```

If excursion:

```
1
2 △ EXCURSION DETECTED
3
4 The reading was saved and the cold-chain
5 alert workflow has been triggered.
6
```

Very similar to the green AWS notification bar you showed, except excursions will use a strong warning treatment.

And importantly, we'll use:

```
1 △ EXCURSION
```

as **text + icon**, not just red colour.

That satisfies the accessibility requirement.

EDIT — PATCH

We'll expose the functionality you added during Week 3.

For example:

```
1 Branch ID
2 [ BRANCH-001 ]
3
4 Reading UUID
5 [ a82c... ]
6
7 Fields to update
8
9 Minimum °C
10 [      ]
11
12 Maximum °C
13 [      ]
14
15 Recorded By
16 [      ]
17
18 [ Update Reading ]
```

This calls:

```
1 PATCH
2 /branches/{branchId}/readings/{readingId}
```

If an edit changes a safe reading into an excursion, we'll show the excursion notification as well.

DELETE

Something like:

```
1 Branch ID
2 [ BRANCH-001 ]
3
4 Reading UUID
5 [ a82c... ]
6
7 [ Delete Reading ]
```

and before deletion:

```
1 Are you sure you want to delete this reading?
2
3 Reading:
4 a82c...
5
6 [ Cancel ] [ Delete Reading ]
```

We're not going to make DELETE too easy to accidentally press.

This will be the main database-style screen.

At the top:

1

Temperature Readings

2

3

View:

4

[Branch Readings ▼]

← selector

5

6

Branch:

7

[BRANCH-001 ▼]

8

9

[Refresh]

The selector will have:

1

Branch Readings

2

Individual Reading

So this single page satisfies both forms of GET.

Branch mode

It will display a proper table:

UUID	Branch	Date	Time	Fridge	Min °C	Max °C	Status	Alert Raised At	Recorded By
UUID ...	BRA NCH-001	08/09/26	12:30	FRIDGE-01	3	7	OK	—	Umar
UUID ...	BRA NCH-001	08/09/26	11:22	FRIDGE-02	1.5	9	⚠ EXCURSION	11:22:03	Umar

We'll keep:

1

readingId

completely separate from:

1

recordedAt

just as you requested.

We'll also split `recordedAt` visually into:

```
1 | Date
2 | Time
```

even though the backend stores it as one ISO timestamp.

Excursion rows

An excursion won't simply become red.

We'll use:

```
1 | △ EXCURSION
```

plus visual highlighting.

For example:

```
1 |
2 | △ EXCURSION | 1.5°C | 9°C | Alert: 08/09/2026 11:22 |
3 |
```

The **Status** and **Alert Raised At** headers can also have a light warning/red treatment like you requested.

Individual-reading mode

Selecting:

```
1 | Individual Reading
```

will change the controls to:

```
1 | Branch
2 | [ BRANCH-001 ]
3 |
4 | Reading UUID
5 | [ 550e8400-e29b-41d4-a716-446655440000 ]
6 |
7 | [ Find Reading ]
```

Then we display **every field**:

```
1 | Reading Details
2 |
3 | Reading UUID
4 | 550e8400-e29b-41d4-a716-446655440000
5 |
6 | Branch
7 | BRANCH-001
8 |
9 | Fridge
10 | FRIDGE-001
11 |
12 | Recorded At
13 | 08 September 2026, 12:30
14 |
```

```
15 Minimum Temperature
16 1.5°C
17
18 Maximum Temperature
19 9°C
20
21 Status
22 △ EXCURSION
23
24 Recorded By
25 Umar
26
27 Alert Raised At
28 08 September 2026, 12:30:04
```

This satisfies the ticket's **Reading Detail screen requirement** while still keeping your site to the three main pages you want.

TanStack Query

This is going to control communication with the backend.

Instead of doing things like:

```
1 useEffect(() => {
2   fetch(...)
3 }, []);
```

we'll use TanStack Query.

Conceptually:

```
1 GET
2 ↓
3 useQuery()
4 ↓
5 cache results
6 loading state
7 error state
8 automatic refetching
9 retry behaviour
```

For POST/PATCH/DELETE:

```
1 useMutation()
```

The key distinction is:

```
1 QUERY
2 reads data
3 GET
4
5 MUTATION
6 changes data
7 POST / PATCH / DELETE
```

After a POST, PATCH or DELETE we'll do something like:

```
1 queryClient.invalidateQueries({
```

```
2   queryKey: ["readings", branchId],  
3   });
```

That tells TanStack Query:

The cached readings may now be out of date. Fetch them again.

This is how we'll satisfy:

new reading appears without manually refreshing.

If we forgot invalidation, the database could successfully contain the new record while the browser continued displaying its **old cached list**.

All four UI states

We're going to deliberately build each state rather than treating them as afterthoughts.

Loading

Not:

```
1 | Loading...
```

floating randomly on a blank screen.

We'll give the table a clear loading state:

```
1 | Loading temperature readings...
```

with a visible indicator while keeping the surrounding page structure.

Error

Not:

```
1 | Something went wrong
```

Instead:

```
1 | Unable to load temperature readings.  
2 |  
3 | The API returned an error.  
4 |  
5 | [ Try Again ]
```

Where appropriate we'll show the API's actual validation message.

Empty

Exactly as the ticket asks:

```
1 | No readings recorded for this branch yet.
```

Possibly with:

```
1 | [ Log First Reading ]
```

Success

Tables/data appear normally.

Validation

We'll mirror the backend rules rather than create different frontend rules.

For example:

```
1 | recordedAt
2 | Required
3 | Must be a valid date/time
4 | Cannot be in the future
5 |
6 | minTempC
7 | Required
8 | Must be a number
9 |
10 | maxTempC
11 | Required
12 | Must be a number
13 | Must not be lower than minTempC
14 |
15 | recordedBy
16 | Required
17 | Cannot be empty
```

And once `fridgeId` is added:

```
1 | fridgeId
2 | Required
3 | Must be a non-empty string
```

But the important concept is:

```
1 | Frontend validation
2 |   +
3 | Backend validation
```

not:

```
1 | Frontend validation instead of backend validation
```

We'll still send an invalid request through Postman at the end and prove Lambda rejects it.

CORS

You absolutely are going to encounter this because the browser will be:

```
1 localhost:3000
2   ↓
3 AWS execute-api URL
```

Those are different origins.

Also, you're sending things such as:

```
1 Content-Type: application/json
2 x-api-key
3 PATCH
4 DELETE
```

which makes browser preflight particularly relevant.

When we hit it, we're going to fix the **Terraform backend**, not click around manually in API Gateway.

We'll need CORS support for approximately:

```
1 Origin:
2 http://localhost:3000
3
4 Methods:
5 GET
6 POST
7 PATCH
8 DELETE
9 OPTIONS
10
11 Headers:
12 Content-Type
13 x-api-key
```

And we'll document:

I encountered CORS when the Next.js frontend attempted to call the AWS API Gateway endpoint from localhost. The browser blocked the cross-origin request. I added the appropriate CORS configuration to the Terraform-managed API, including OPTIONS/preflight handling and the required allowed methods and headers.

We'll do the actual implementation when we reach it.

One security point I want to flag

Your API currently uses:

```
1 x-api-key
```

If we put that API key into:

then **anyone using the frontend can inspect it in their browser**.

API Gateway API keys are useful for things like usage control, but they are **not user authentication**.

For this internship project, we can still use it so that we work directly against the API and demonstrate CORS as the ticket intends.

For a real production operations portal I would instead put proper authentication in front of it, such as an authorizer/Cognito, rather than treating an API key embedded in frontend JavaScript as a secret.

I don't want us silently teaching the wrong security pattern.

Project structure we're heading towards

I suggest:

```
1 4week-project-frontend/
2 |
3 |   app/
4 |   |   components/
5 |   |   |   Header.tsx
6 |   |   |   ReadingCard.tsx
7 |   |   |   ReadingsTable.tsx
8 |   |   |   StatusBadge.tsx
9 |   |   |   AlertNotification.tsx
10 |   |
11 |   |   manage/
12 |   |   |   page.tsx
13 |   |
14 |   |   readings/
15 |   |   |   page.tsx
16 |   |
17 |   |   globals.css
18 |   |   layout.tsx
19 |   |   page.tsx
20 |   |   providers.tsx
21 |
22 |   lib/
23 |   |   api.ts
24 |
25 |   types/
26 |   |   reading.ts
27 |
28 |   public/
29 |
30 |   .env.local
31 |   package.json
32 |   ...
```

That is still small enough to understand, but we're no longer going to put the whole application into three giant `page.tsx` files.